

Aplicación de Técnicas Avanzadas de IA en Modelos de Lenguaje

Bradley Campo Hurtado- Camilo Velasco- Antonio Dorado- Simon David Colmenares

Resumen—This paper presents the design and implementation of a corporate question answering (Q&A) system for Smurfit Kappa Colombia, evolved across two phases. Phase 1 builds the system upon the Retrieval-Augmented Generation (RAG) paradigm in three variants: keyword-based lexical search, semantic retrieval with FAISS over local Ollama embeddings, and a commercial variant combining Google Gemini embeddings with Moonshot AI Kimi K2.6 for generation. Phase 2 evolves the system into a conversational agent with persistent memory and a tool router: the LLM autonomously decides between two information sources — RAG over the documental knowledge base, or a deterministic structured-data tool curated from JSON. The agent is implemented with LangChain’s `create_tool_calling_agent` and exposes its execution in real time through Server-Sent Events, including tool invocation events for UI transparency. A robust four-level fallback system handles edge cases where smaller open-source models fail to produce final responses after tool calls. The Phase 2 system is dockerized as a single multi-stage container (Node build → Python serve), deployed to a VPS via Easypanel at `llm.innovatec.co`, and instrumented with LangSmith for full observability of agent execution traces. The frontend is a React + Vite + Tailwind chat UI with streaming, per-user history, and configurable sampling parameters. Both phases prioritize answer grounding through carefully engineered prompts that prohibit reliance on pre-training knowledge, mitigating hallucinations over sensitive corporate information.

Index Terms—Retrieval-Augmented Generation (RAG), LLM agents, tool calling, conversational memory, modelos de lenguaje de gran escala (LLM), búsqueda semántica, FAISS, embeddings, LangChain, Ollama, Google Gemini, Kimi K2.6, Moonshot AI, OpenAI, OpenRouter, FastAPI, React, Docker, Easypanel, LangSmith, Server-Sent Events.

I. INTRODUCTION

El presente informe documenta el diseño, implementación y análisis de un sistema de preguntas y respuestas (*Question Answering*, Q&A) basado en la técnica de *Retrieval-Augmented Generation* (RAG) para Smurfit Kappa Colombia, empresa líder en empaques de papel y cartón corrugado en el país.

II. MOTIVACIÓN

Los grandes modelos de lenguaje (LLMs) poseen conocimiento general amplio, pero carecen de información actualizada y específica sobre organizaciones particulares. En el caso de Smurfit Kappa Colombia, datos como ubicaciones de plantas, productos ofrecidos, historia corporativa y compromisos de sostenibilidad no suelen estar correctamente representados en el conocimiento de preentrenamiento de los modelos, o pueden encontrarse desactualizados.

Para resolver esto el sistema implementado sigue dos principios fundamentales:

1. **Soberanía de datos:** todos los componentes modelos, embeddings e índice vectorial se ejecutan de forma completamente local mediante Ollama y FAISS.
2. **Fundamentación en fuentes oficiales:** el modelo responde exclusivamente a partir de fragmentos extraídos del sitio web oficial de la empresa, eliminando alucinaciones sobre información corporativa.

II-A. Objetivos del Sistema

- Generar automáticamente una base de conocimiento desde el volcado completo del sitio web (`full_dump.json`).
- Proveer una interfaz de consulta que recupere los fragmentos más relevantes para cada pregunta y los proporcione al LLM como contexto.
- Ofrecer dos modalidades complementarias: búsqueda léxica (*keyword*) y búsqueda semántica mediante embeddings y FAISS.
- Exponer funcionalidades adicionales: resumen ejecutivo y generación de FAQs.

III. ARQUITECTURA GENERAL DEL SISTEMA

El sistema está compuesto por cuatro módulos principales que actúan en pipeline:

IV. MÓDULO 1 — GENERACIÓN DE LA BASE DE CONOCIMIENTO (`BUILD_KB.PY`)

Este módulo lee el volcado completo del sitio web y produce dos variantes de la base de conocimiento en Markdown:

- **`knowledge_base.md`** (versión compacta): destinada a `app.py`. Limitada a 6000 palabras para caber íntegramente en el contexto de modelos pequeños.
- **`knowledge_base_rag.md`** (versión completa): destinada a `app_rag.py`. Sin límite de palabras, ya que el LLM solo procesa los *chunks* recuperados, no el documento completo.

IV-A. Módulo 2 — Q&A por Palabras Clave (`app.py`)

Implementa un motor de recuperación léxico. Dado que el documento completo cabe en el contexto del modelo, se evita la complejidad de embeddings y se priorizan los párrafos cuyas palabras coincidan con la pregunta del usuario.

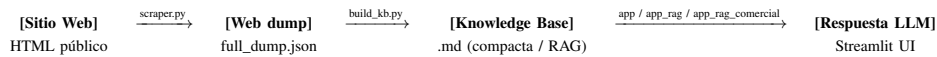


Figura 1. Flujo de datos de extremo a extremo, desde el *crawl* del sitio web hasta la respuesta del LLM.

IV-B. Módulo 3 — Q&A con RAG Semántico (*app_rag.py*)

Implementa el paradigma RAG completo usando LangChain, FAISS y embeddings de Ollama. Divide la base de conocimiento en *chunks*, genera vectores semánticos para cada uno, y en tiempo de consulta recupera los k fragmentos más similares a la pregunta para construir el contexto del LLM.

V. MÓDULO DE SCRAPING DEL SITIO OFICIAL (*SCRAPER.PY*)

*Nota: el scraping es la primera etapa del pipeline, previa a todos los módulos descritos anteriormente. Aquí se documenta cómo se genera *full_dump.json*, archivo que actúa como entrada para *build_kb.py*.*

V-A. Visión General

El módulo *scraper.py* implementa un *crawler* asíncrono que descarga la totalidad del sitio web oficial de Smurfit Kappa Colombia (<https://www.smurfitkappa.com/co>) y produce el archivo *full_dump.json* que sirve como entrada para *build_kb.py*. Sin este módulo, todo el resto del pipeline carece de fuente de datos.

El scraper está diseñado para ser respetuoso con el sitio (limita la concurrencia, espera entre peticiones, honra *robots.txt*) y robusto ante fallos transitorios (reintento con *backoff* exponencial, guardado incremental).

V-B. Configuración del Crawl

Cuadro I
PARÁMETROS DE OPERACIÓN DEL SCRAPER.

Parámetro	Valor
BASE_URL	/co de smurfitkappa.com
CONCURRENCY	10 peticiones simultáneas
DELAY_SECONDS	0.5s entre peticiones por <i>worker</i>
MAX_PAGES	5000 páginas
REQUEST_TIMEOUT	20s por petición
MAX_RETRIES	3 reintentos por URL
RETRY_BACKOFF	2s base (× intento)
SAVE EVERY	200 páginas (guardado incremental)

V-C. Crawler Asíncrono con *aiohttp*

Se utiliza *asyncio* junto con *aiohttp* para realizar peticiones HTTP concurrentes, lo que reduce drásticamente el tiempo total respecto a un crawler secuencial. La concurrencia está acotada por un Semaphore para evitar saturar el servidor:

Listing 1. Núcleo del crawler asíncrono.

```

1 semaphore = asyncio.Semaphore(CONCURRENCY)
2 async with ...
    aiohttp.ClientSession(headers=HEADERS,
3                               connector=connector,
4                               as session:
    while queue and len(visited) < MAX_PAGES:
  
```

```

5 batch = pop_unvisited(queue, batch_size)
6 tasks = [fetch_page(session, url, ...
7             semaphore, loop)
8           for url in batch]
9 results = await asyncio.gather(*tasks)
10 for doc in results:
11     if doc:
12         documents.append(doc)
13         queue.extend(doc["internal_links"])
  
```

El parseo HTML se ejecuta en un *thread pool* mediante *loop.run_in_executor* para no bloquear el *event loop* durante operaciones de CPU intensivas.

V-D. Honrar *robots.txt*

Antes de iniciar el *crawl*, se carga el archivo *robots.txt* del dominio mediante *urllib.robotparser.RobotFileParser*. Cada URL candidata pasa por una verificación *robots.can_fetch* antes de ser solicitada, descartando aquellas explícitamente bloqueadas para el *User-Agent* configurado.

V-E. Reintento con *Backoff* Exponencial

Las peticiones que devuelven HTTP 429 (*Too Many Requests*) o errores de red transitorios se reintentan hasta *MAX_RETRIES*=3 veces con tiempos de espera crecientes:

Listing 2. Manejo de *rate limit* en el crawler.

```

1 if resp.status == 429:
2     wait = RETRY_BACKOFF * attempt # 2s, ...
3     4s, 6s
4     await asyncio.sleep(wait)
5     continue
  
```

Esta estrategia es complementaria a la utilizada después en la generación de *embeddings* con Gemini, lo que muestra que el patrón de *backoff* es transversal a todo el pipeline.

V-F. Filtrado de Contenido

Se descartan automáticamente las URLs que no corresponden a páginas HTML:

- **Por extensión:** PDFs, imágenes (.jpg, .png, .svg), audios, vídeos, documentos ofimáticos, archivos comprimidos y recursos estáticos (.css, .js, fuentes).
- **Por Content-Type:** solo se procesan respuestas cuyo encabezado contenga *text/html*.
- **Por dominio:** se restringe a enlaces internos del subpath /co, evitando seguir enlaces a otros países o sitios externos.

V-G. Parseo y Extracción Estructurada

Para cada página HTML se ejecuta BeautifulSoup con el parser *lxml* y se extraen los siguientes campos estructurados:

Cuadro II
CAMPOS EXTRAÍDOS POR PÁGINA.

Campo	Origen
title	<title>
content	Texto limpio (sin nav, footer, etc.)
headings	h1-h6
metadata	description, keywords, OG, idioma
images	src, alt, title
external_links	enlaces fuera del dominio
internal_links	enlaces internos (cola del crawler)
chunks	particiones del contenido para RAG
http_status, word_count	metadatos de auditoría

Antes de extraer texto, se eliminan etiquetas que típicamente contienen ruido de navegación: script, style, noscript, nav, footer, header, aside, form, button, input, etc. Esta limpieza estructural es complementaria a la limpieza léxica posterior en `build_kb.py`.

V-H. Chunking en el Origen

Aunque el *chunking* definitivo para RAG se realiza más tarde con `RecursiveCharacterTextSplitter` de `LangChain`, el scraper también produce una versión preliminar de *chunks* (`max_chars=1500`, `overlap=200`) en el archivo `pages.jsonl`. Esto permite usar el *dump* directamente en otras pipelines (por ejemplo, indexación en bases vectoriales externas) sin reprocesar el HTML.

V-I. Guardado Incremental

Para no perder progreso ante caídas, el scraper persiste a disco cada `SAVE_EVERY=200` páginas:

- `output/pages.jsonl`: una línea JSON por *chunk*, lista para ingesta de *embeddings*.
- `output/full_dump.json`: volcado completo con estadísticas agregadas (entrada principal de `build_kb.py`).
- `output/url_index.txt`: índice legible de URLs visitadas con título y *word count*.
- `output/scrape_log.txt`: log de ejecución para auditoría y depuración.

V-J. Resultados del Crawl

La ejecución sobre el sitio oficial produjo los siguientes resultados:

Cuadro III
ESTADÍSTICAS DEL CRAWL SOBRE `SMURFITKAPPA.COM/CO`.

Métrica	Valor
Páginas indexadas	1 401
Chunks preliminares generados	4 079
URLs únicas visitadas	1 401
Tiempo total del crawl	318.9 s (~5 min)
Concurrencia efectiva	10 workers

El tiempo total del crawl (~5 minutos) es notablemente bajo para un volumen de 1 401 páginas, gracias al diseño asíncrono. Una implementación secuencial equivalente, asumiendo ~0.3 s por petición y respetando `DELAY_SECONDS=0.5`, habría tardado más de 20 minutos.

VI. GENERACIÓN DE LA BASE DE CONOCIMIENTO

VI-A. Proceso de Limpieza y Filtrado

El archivo `full_dump.json` contiene todas las páginas del sitio web de Smurfit Kappa Colombia. El script `build_kb.py` aplica los siguientes pasos de preprocesamiento:

VI-A1. Filtrado por longitud mínima: Se descartan páginas con menos de 80 palabras (`MIN_WORDS = 80`), ya que corresponden a páginas de navegación o redirecciones sin contenido sustantivo.

VI-A2. Deduplicación por URL normalizada: Las URLs se normalizan eliminando parámetros de consulta y fragmentos, y forzando el esquema `https`. Esto evita indexar la misma página bajo variantes de URL.

Listing 3. Normalización de URLs para deduplicación.

```
1 def normalize_url(url: str) -> str:
2     parsed = urlparse(url)
3     return urlunparse(
4         ("https", parsed.netloc, ...
5         parsed.path, "", "", ""))
6     ).rstrip("/")
```

VI-A3. Limpieza de ruido de navegación: Se define un conjunto `NAV_NOISE` con más de 30 cadenas de texto que corresponden a elementos de menú, botones y etiquetas de navegación (e.g., “Leer más”, “Expandir ícono”, “Filtrar por país”). Estas líneas se eliminan durante el procesamiento de cada página.

VI-A4. Encabezado estático enriquecido: La constante `HEADER` inyecta al inicio de ambos archivos una tabla curada manualmente con datos clave de la empresa (fechas de fundación, fusiones, cifras globales) y una lista estructurada de direcciones de todas las plantas en Colombia. Este contenido garantiza que las consultas factuales más frecuentes siempre tengan contexto disponible, independientemente de los resultados de recuperación.

VI-B. Priorización y Secciones

Las páginas se clasifican según patrones de URL en tres secciones:

Cuadro IV
PATRONES DE SECCIÓN PARA LA ORGANIZACIÓN DEL KB.

Patrón de URL	Sección	Límite RAG
<code>/co/about</code>	Sobre la Empresa	Todas las páginas
<code>/co/locations</code>	Ubicaciones y Plantas	Todas las páginas
<code>/co/products-and-services</code>	Productos y Servicios	Top-25 por word count

Dentro de cada sección, las páginas se ordenan por número de palabras (descendente), asegurando que los contenidos más ricos se indexen primero.

VI-C. Distinción entre Versión Compacta y RAG

Cuadro V
COMPARACIÓN ENTRE LAS DOS VARIANTES DEL KNOWLEDGE BASE.

Característica	Compacta (app.py)	RAG (app_rag.py)
Límite de palabras	6000	Sin límite
Estrategia de contexto	Documento completo	Chunks recuperados
Página raíz de ubicaciones	Incluida	Excluida (skip)
Límite de productos	Sin límite	Top-25 por extensión
Motor de recuperación	Léxico (keywords)	Semántico (FAISS)

La exclusión de la página raíz de ubicaciones (/co/locations) en la versión RAG responde a que su contenido es un resumen mezclado de todas las plantas; en RAG se prefieren las páginas individuales por planta, más específicas y sin redundancia.

VII. SISTEMA Q&A POR PALABRAS CLAVE (APP.PY)

VII-A. Recuperación Léxica

La función `retrieve_context` implementa un motor de recuperación basado en coincidencia de tokens. El proceso es:

leftmargin=2cm

1. Se extrae siempre la tabla “Datos Clave” del inicio del KB, asegurando que el LLM disponga de los hechos corporativos fundamentales.
2. El texto del KB se divide en párrafos (bloques separados por líneas vacías).
3. La pregunta del usuario se tokeniza, eliminando *stop-words* en español (artículos, preposiciones y pronombres interrogativos).
4. Cada párrafo recibe un puntaje según cuántos tokens de la pregunta aparecen en él.
5. Los párrafos se ordenan por puntaje y se acumulan en el contexto hasta alcanzar el límite de 3000 caracteres.

Listing 4. Función de puntuación léxica por párrafo.

```
1 def score(para: str) -> int:
2     lower = para.lower()
3     return sum(1 for w in q_words if w in lower)
4
5 scored = sorted(paragraphs, key=score, ...
                  reverse=True)
```

VII-B. Prompt de Sistema

El `SYSTEM_PROMPT` instruye al LLM con reglas estrictas:

- Usar *exclusivamente* el fragmento proporcionado.
- Si el contexto no contiene la respuesta, declararlo explícitamente.
- **No** usar conocimiento previo del modelo sobre la empresa.
- Responder siempre en español.

Este diseño de prompt mitiga el riesgo de alucinaciones, que es especialmente relevante cuando el modelo tiene información de preentrenamiento incorrecta sobre la empresa.

VII-C. Streaming de Respuestas

La generación se realiza en modo *streaming* mediante `ollama.chat(..., stream=True)`, actualizando el componente `st.empty()` de Streamlit con cada token. Se aplica la función `strip_thinking` para eliminar bloques `<think>...</think>` generados por modelos con razonamiento encadenado (*chain-of-thought* como Qwen3), garantizando que el usuario solo vea la respuesta final.

Listing 5. Eliminación de bloques de razonamiento interno.

```
1 def strip_thinking(text: str) -> str:
2     text = re.sub(r"<think>.*?</think>", "", ...
3         text, flags=re.DOTALL)
4     text = re.sub(r"<think>.*$", ...
5         "", text, flags=re.DOTALL)
6     return text.strip()
```

VIII. SISTEMA Q&A CON RAG SEMÁNTICO (APP_RAG.PY)

VIII-A. Arquitectura RAG

El módulo `app_rag.py` implementa el patrón RAG canónico usando LangChain como orquestador. El flujo de una consulta es:

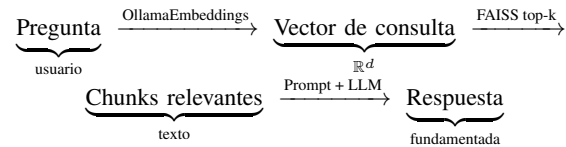


Figura 2. Pipeline de inferencia RAG.

VIII-B. División en Chunks

Se utiliza `RecursiveCharacterTextSplitter` de LangChain con los siguientes parámetros:

Cuadro VI
PARÁMETROS DE SEGMENTACIÓN DEL KNOWLEDGE BASE RAG.

Parámetro	Valor
<code>chunk_size</code>	1500 caracteres
<code>chunk_overlap</code>	200 caracteres
<code>separators</code>	<code>[\n##, \n###, \n\n, \n, " "]</code>

Los separadores están ordenados por prioridad semántica: primero se intenta dividir en secciones de Markdown (##), luego subsecciones (###), luego párrafos, líneas y palabras. El solapamiento de 200 caracteres preserva el contexto en los bordes de cada fragmento.

VIII-C. Índice Vectorial FAISS

VIII-C1. Generación del índice: Los embeddings se generan mediante `OllamaEmbeddings` con el modelo `nomic-embed-text-v2-moe:latest`. El índice FAISS se persiste en disco (`output/faiss_index/`) para evitar regenerarlo en cada arranque de la aplicación.

Listing 6. Lógica de carga o creación del índice FAISS.

```

1  @st.cache_resource(show_spinner="Construyendo ...
   ndice vectorial...")
2  def load_vectorstore():
3      embeddings = get_embeddings()
4      if FAISS_PATH.exists():
5          return FAISS.load_local(
6              str(FAISS_PATH), embeddings,
7              allow_dangerous_deserialization=True,
8              )
9      # Primera ejecución: generar embeddings ...
   y guardar
10     kb_text = ...
11     KB_PATH.read_text(encoding="utf-8")
12     chunks = splitter.split_text(kb_text)
13     docs = [Document(page_content=c for ...
   c in chunks)]
14     vectorstore = FAISS.from_documents(docs, ...
   embeddings)
15     FAISS_PATH.mkdir(parents=True, ...
   exist_ok=True)
16     vectorstore.save_local(str(FAISS_PATH))
   return vectorstore

```

VIII-C2. Recuperación semántica: En tiempo de consulta, el retriever de FAISS selecciona los k chunks con mayor similitud coseno al vector de la pregunta. El parámetro k es configurable por el usuario en la barra lateral (rango 1–8, predeterminado 4).

[Ventaja de la búsqueda semántica] A diferencia de la búsqueda léxica, el sistema RAG puede recuperar fragmentos relevantes aunque la pregunta use sinónimos o formulaciones distintas a las del documento. Por ejemplo, “¿Dónde producen sus cajas?” puede recuperar fragmentos que contienen “plantas corrugadoras” sin que ninguna de esas palabras aparezca en la pregunta.

VIII-D. Construcción del Prompt RAG

El contexto se construye concatenando los k chunks con separadores --:

Listing 7. Formateo del contexto multi-chunk para el LLM.

```

1  def format_docs(docs):
2      return "\n\n---\n\n".join(d.page_content ...
   for d in docs)

```

La cadena LangChain (*chain*) combina el retriever, el formateador, el prompt y el LLM en un pipeline declarativo:

Listing 8. Definición de la cadena RAG con LangChain LCEL.

```

1  chain = (
2      {"context": retriever | format_docs,
3      "question": RunnablePassthrough()}
4      | QA_PROMPT
5      | llm
6      | StrOutputParser()
7  )

```

VIII-E. Funcionalidades Adicionales

VIII-E1. Resumen Ejecutivo: Se usa SUMMARY_PROMPT para instruir al LLM a generar un resumen estructurado de hasta 300 palabras cubriendo: identidad corporativa, productos, ubicaciones, valores y presencia global. El documento de entrada se trunca a los primeros 6000 caracteres del KB para evitar superar el contexto.

VIII-E2. Generación de FAQs: El FAQ_PROMPT solicita exactamente 10 preguntas frecuentes con sus respuestas, en formato P: ... / R: ..., cubriendo temas variados: historia, productos, ubicaciones, sostenibilidad y contacto. Esta funcionalidad es útil para poblar portales de soporte o materiales de onboarding.

IX. MÓDULO 4 — RAG CON APIS COMERCIALES

(APP_RAG_COMERCIAL.PY)

IX-A. Motivación

Si bien la versión local (app_rag.py) garantiza soberanía total de datos y costos fijos, presenta dos limitaciones intrínsecas: (i) la calidad de los modelos ejecutables en hardware local está acotada por la memoria y cómputo disponibles, y (ii) el contexto efectivo de los modelos abiertos suele ser sensiblemente menor al de los modelos comerciales de frontera.

Para evaluar el trade-off entre soberanía de datos y calidad de modelo se implementa una tercera variante que sustituye la inferencia local por APIs comerciales, manteniendo el resto de la arquitectura idéntica.

IX-B. Arquitectura Híbrida

En esta variante solo el almacén vectorial sigue siendo local (FAISS persistido en disco). Tanto la generación de embeddings como la generación de respuestas se delegan a proveedores comerciales:

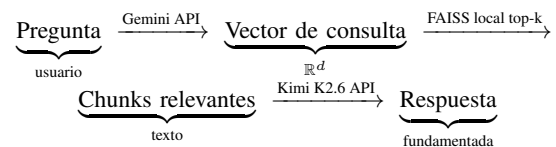


Figura 3. Pipeline RAG híbrido con APIs comerciales.

Una propiedad arquitectónica relevante de esta solución es que la cadena LCEL de LangChain es *idéntica* a la de la versión local: solo cambian las clases concretas de embeddings y LLM. Esto valida la *portabilidad multi-proveedor* como una característica directa de usar LangChain como orquestador.

IX-C. Modelo de Embeddings: Google Gemini

Se selecciona el modelo gemini-embedding-2-preview de Google, accedido mediante la clase GoogleGenerativeAIEmbeddings de langchain-google-genai 4.x. Este modelo se eligió por:

- Calidad multilingüe SOTA, particularmente en español, lo que beneficia directamente la recuperación de fragmentos en preguntas formuladas en ese idioma.
- Soporte nativo para tareas de *retrieval* con tipos de tarea explícitos (RETRIEVAL_QUERY, RETRIEVAL_DOCUMENT).
- Disponibilidad en el *Free Tier* de Google AI Studio para prototipado, con escalado a *Paid Tier* para producción.

IX-D. Modelo LLM: Moonshot AI Kimi K2.6

Para la generación se utiliza `kimi-k2.6` de Moonshot AI, accedido mediante `ChatMoonshot` de `langchain-moonshot`. Las razones principales son:

- **Contexto extendido de 256K tokens**, que permite enviar un número mucho mayor de chunks recuperados sin truncamiento, eliminando el cuello de botella de los modelos locales (~8K-32K tokens).
- **Calidad de frontera**: en *benchmarks* como SWE-Bench compite con los modelos comerciales más avanzados, ofreciendo respuestas más precisas en preguntas complejas.
- **Modo *thinking* opcional**, que habilita razonamiento encadenado interno para preguntas que requieren combinar múltiples fragmentos.
- **API compatible con OpenAI**, lo que facilita la integración sin requerir un cliente especializado.

Una restricción técnica relevante de `kimi-k2.6` es que el parámetro `temperature` está fijo en `1.0` por diseño del proveedor: el servicio rechaza con un error `400 Bad Request` cualquier valor distinto. Esto rompe la convención habitual de usar `temperature=0.0` para respuestas deterministas en sistemas Q&A corporativos. La mitigación de variabilidad recae entonces en el diseño del *prompt* (instrucciones estrictas de fundamentación) más que en el parámetro de muestreo.

IX-E. Implementación

La cadena LCEL de la versión comercial es estructuralmente equivalente a la local; solo cambian los componentes concretos:

Listing 9. Inicialización de embeddings y LLM comerciales.

```
1 from langchain_google_genai import ...
   GoogleGenerativeAIEmbeddings
2 from langchain_moonshot import ChatMoonshot
3
4 embeddings = GoogleGenerativeAIEmbeddings(
5     model="models/gemini-embedding-2-preview",
6     google_api_key=os.getenv("GOOGLE_API_KEY"),
7 )
8
9 llm = ChatMoonshot(
10     model="kimi-k2.6",
11     temperature=1.0,    # restricción del modelo
12     max_retries=2,
13 )
```

El índice FAISS se persiste en `output/faiss_index_gemini/` para diferenciarlo del índice generado con embeddings de Ollama, ya que los vectores de proveedores distintos no son intercambiables (distinta dimensionalidad y espacio semántico).

X. DESAFÍOS DE INTEGRACIÓN CON APIS COMERCIALES

La integración con servicios comerciales introdujo varios retos prácticos que no aparecen en el escenario puramente local. Documentarlos es relevante porque condicionan directamente la robustez del sistema en producción.

X-A. Cuotas y Rate Limiting

El *Free Tier* de la API de Gemini impone límites estrictos de *requests* por minuto, especialmente para modelos en estado *preview*. La generación inicial del índice FAISS requiere *embeddings* para cientos de *chunks*, lo que supera fácilmente la cuota disponible y produce errores `429 RESOURCE_EXHAUSTED`.

La solución adoptada combina dos estrategias:

1. **Habilitar *billing* en Google Cloud**, lo que eleva las cuotas de forma sustancial y elimina los rechazos en condiciones normales de uso. Esto representa el primer punto donde el sistema deja de ser gratuito.
2. ***Exponential backoff* en la capa de aplicación**, como red de seguridad ante picos transitorios. Ante un `429` se reintenta hasta tres veces con tiempos de espera crecientes (4s, 8s, 16s).

Listing 10. Reintento con *backoff* exponencial frente a 429.

```
1 for intento in range(MAX_RETRIES):
2     try:
3         all_vectors.extend(embeddings.embed_documents(lote))
4         break
5     except Exception as e:
6         if "429" in str(e) or ...
           "RESOURCE_EXHAUSTED" in str(e):
7             wait = SLEEP_BETWEEN_BATCHES * ...
               (2 ** intento)
8             time.sleep(wait)
9         else:
10            raise
```

X-B. Batching de Embeddings

La API de Gemini limita el número de *chunks* por petición y la implementación de `embed_documents` de `LangChain` no maneja automáticamente el particionado para corpus grandes. Se implementa un *batching* manual con tamaño de lote `BATCH=20` y una pausa de cortesía de un segundo entre lotes para suavizar el perfil de carga sobre la API. Una barra de progreso de `Streamlit` informa al usuario el avance del proceso.

X-C. Restricción de Temperatura en Kimi K2.6

Como se mencionó, `kimi-k2.6` solo admite `temperature=1.0`. El intento de pasar `0.0` (estándar para Q&A determinista) o `0.6` resulta en:

```
openai.BadRequestError: 400 - invalid temperature:
only 1.0 is allowed for this model
```

Esto implica un cambio de paradigma respecto a la versión local: el determinismo en las respuestas debe garantizarse mediante el diseño del *prompt* (instrucciones estrictas, prohibición explícita de conocimiento externo, formato de respuesta estructurado) en lugar de mediante el control fino del parámetro de muestreo.

XI. ANÁLISIS COMPARATIVO: LOCAL VS. COMERCIAL

La siguiente tabla complementa la comparación previa entre búsqueda léxica y RAG semántico, contrastando ahora el

RAG semántico local con el RAG semántico basado en APIs comerciales:

Cuadro VII
COMPARACIÓN ENTRE LA VARIANTE LOCAL Y LA VARIANTE COMERCIAL DEL RAG.

Criterio	Local (app_rag.py)	Comercial (app_rag_comercial.py)
Embeddings	nomic-embed-text-v2-moe	gemini-embedding-2-preview
LLM generativo	gemma3:12b, mistral-nemo	kimi-k2.6
Contexto del LLM	~8K-32K tokens	256K tokens
Soberanía de datos	Total	Datos enviados a Google y Moonshot
Modelo de costos	Hardware (fijo)	Por token (variable)
Latencia	Depende de GPU local	Depende de red y proveedor
Cuotas / rate limits	Ninguno	Sí (estrictos en <i>Free Tier</i>)
Determinismo	temperature=0.0	temperature=1.0 forzado
Operación offline	Sí	No
Calidad esperada	Media-alta	Frontera

La existencia de las tres variantes (app.py, app_rag.py, app_rag_comercial.py) define un espacio de configuración que cubre prácticamente todos los escenarios de despliegue: desde entornos sin GPU y con datos altamente confidenciales (léxico local), hasta despliegues que priorizan la calidad máxima de respuesta y aceptan latencia y costo de red (RAG comercial).

XII. INTEGRACIÓN DE MODELOS Y SELECCIÓN DINÁMICA

XII-A. Modelos LLM

Ambas aplicaciones consultan la API local de Ollama para listar los modelos disponibles y filtran aquellos que sean de tipo *embedding*:

Listing 11. Filtrado de modelos de lenguaje generativo.

```
1 EMBEDDING_FAMILIES = {"nomic-bert-moe", ...
2   "bert", "nomic"}
3
4 def get_local_models() -> list[str]:
5     return [
6         m.model for m in ollama.list().models
7         if not any(f in (m.details.family or "")
8                   for f in EMBEDDING_FAMILIES)
9         and "embed" not in m.model.lower()
10    ]
```

Se define un orden de preferencia de modelos para selección automática del predeterminado:

Cuadro VIII
MODELOS UTILIZADOS

Aplicación	Preferencia de modelo
app.py (compacta)	llama3.2:3b, qwen3-fast, qwen3:4b, gemma4:e2b
app_rag.py	gemma3:12b, mistral-nemo, llama3.2:3b, qwen3:4b

La versión RAG puede usar modelos más grandes porque solo procesa los chunks recuperados, mientras que la versión compacta necesita modelos que quepan en contextos de ~4000 tokens.

XII-B. Modelo de Embeddings

El modelo de embeddings nomic-embed-text-v2-moe:latest es un modelo de tipo Mixture-of-Experts optimizado para tareas de recuperación semántica. Opera completamente en local y no transmite datos fuera del entorno.

XIII. INTERFAZ DE USUARIO CON STREAMLIT

Ambas aplicaciones exponen una interfaz web mediante Streamlit con los siguientes elementos:

leftmargin=2cm

- **Barra lateral:** selección de modelo LLM y, en la versión RAG, control deslizante para el parámetro k de recuperación.
- **Área de texto:** entrada de preguntas en lenguaje natural.
- **Expander de chunks:** en app_rag.py, permite inspeccionar los fragmentos exactos que el retriever seleccionó, facilitando la depuración y la explicabilidad del sistema.
- **Streaming de tokens:** la respuesta se renderiza progresivamente, mejorando la percepción de velocidad de respuesta.
- **Pestañas** (solo app_rag.py): Q&A, Resumen ejecutivo y FAQs en tabs separados.

XIV. ANÁLISIS COMPARATIVO: KEYWORD VS. RAG SEMÁNTICO

Cuadro IX
COMPARACIÓN TÉCNICA ENTRE LAS DOS ESTRATEGIAS DE RECUPERACIÓN.

Criterio	Búsqueda (app.py)	Léxica (app_rag.py)	RAG Semántico (app_rag.py)
Velocidad de inicio	Inmediata		Lenta (primera vez: embeddings)
Precisión semántica	Baja (tokens exactos)		Alta (similitud vectorial)
Dependencia de vocabulario	Alta		Baja
Infraestructura	Solo Ollama		Ollama + FAISS + LangChain
Escalabilidad del KB	Limitada (~6k palabras)		Alta (sin límite práctico)
Explicabilidad	Párrafos por puntaje		Chunks recuperados visibles
Temperatura del LLM	0.0 (determinista)		0.0 (determinista)
Recursos de hardware	Mínimos		Moderados (GPU recomendada)

La arquitectura dual permite adaptar el sistema según las restricciones del entorno: en máquinas con pocos recursos se usa app.py; cuando se dispone de GPU o mayor RAM, app_rag.py ofrece recuperación significativamente mejor.

XV. TÉCNICAS DE PROMPT ENGINEERING APLICADAS

El sistema utiliza tres prompts *system* diferentes según la tarea: QA_PROMPT para preguntas y respuestas, SUMMARY_PROMPT para el resumen ejecutivo y FAQ_PROMPT para la generación de preguntas frecuentes. Cada uno combina varias técnicas reconocidas de *prompt engineering* con el objetivo común de fundamentar las respuestas en el contexto recuperado y producir salidas predecibles.

XV-A. Paradigma Zero-Shot

Los tres prompts operan bajo el paradigma *zero-shot*: el modelo recibe únicamente instrucciones y debe generar la respuesta sin ningún ejemplo resuelto en el prompt. Esta es una decisión consciente:

- **Eficiencia de contexto:** incluir ejemplos resueltos consumiría *tokens* valiosos del presupuesto de contexto, especialmente importante para los modelos locales con ventanas más reducidas (~8K–32K tokens).
- **Generalización:** los ejemplos pueden sesgar al modelo hacia el estilo o temática de los ejemplos provistos. En *zero-shot* el modelo se apoya únicamente en su capacidad de seguimiento de instrucciones, lo cual produce salidas más diversas y adaptativas.
- **Mantenibilidad:** las instrucciones son más fáciles de modificar y auditar que un conjunto de ejemplos cuya curación es laboriosa.

Es importante distinguir entre *zero-shot puro* y *zero-shot con esquema de salida*: `FAQ_PROMPT` incluye una plantilla literal del formato esperado (`**P: [pregunta]** / R: [respuesta]`), pero los corchetes contienen *placeholders*, no ejemplos resueltos. Esto se llama **output schema specification** y es estrictamente *zero-shot*, no *few-shot*.

La viabilidad del *zero-shot* en este sistema descansa en dos hechos: (i) los modelos modernos siguen instrucciones complejas con alta fidelidad, y (ii) el contexto recuperado por RAG ya provee la *materia prima* concreta de la respuesta, por lo que el modelo solo necesita reformularla, no inventarla.

XV-B. Técnicas Transversales

Las siguientes técnicas se aplican en los tres prompts:

XV-B1. Role prompting: A cada prompt se le antepone una asignación de persona profesional, lo que condiciona estilo, vocabulario y formalidad sin necesidad de especificarlos explícitamente:

- **QA:** “Eres un asistente virtual de Smurfit Kappa Colombia”.
- **SUMMARY:** “Eres un analista experto en comunicación corporativa”.
- **FAQ:** “Eres un experto en comunicación con clientes”.

XV-B2. Grounding (anclaje al contexto): El núcleo del paradigma RAG. Se obliga al modelo a responder **únicamente** a partir del contenido recuperado, prohibiendo el uso de conocimiento de preentrenamiento. La instrucción se enfatiza visualmente con la palabra **ÚNICAMENTE** en mayúsculas, que el modelo interpreta como una restricción de alta prioridad:

“Responde la pregunta usando **ÚNICAMENTE** los fragmentos de contexto proporcionados.”

XV-B3. Separación system/human: El uso de `ChatPromptTemplate.from_messages` con roles explícitos (`system` y `human`) separa las **instrucciones permanentes** de los **datos variables**. Esto reduce la superficie de ataque por *prompt injection* desde la entrada del usuario y mejora el cumplimiento de las reglas, ya que los modelos modernos priorizan el rol `system` sobre el `human`.

XV-B4. Variable interpolation: Los *placeholders* `{context}`, `{question}` y `{document}` permiten inyectar datos dinámicos manteniendo el prompt fijo. Patrón estándar de *templating* que evita la concatenación manual de cadenas y los errores asociados.

XV-B5. Restricción de idioma: Todos los prompts incluyen “Responde en español”. Es una instrucción aparentemente trivial, pero crítica: los LLMs tienden a derivar al inglés cuando el contexto es técnico o cuando se mezclan idiomas, lo que rompería la experiencia de usuario.

XV-C. Técnicas Específicas por Prompt

XV-C1. QA_PROMPT:

- **Negative constraints (guardrails):** la regla “Nunca uses conocimiento externo sobre la empresa” es un *guardrail* explícito contra alucinaciones, complementario al *grounding*.
- **Fallback instruction:** la cláusula “Si el contexto no contiene la respuesta, di: Esa información no está disponible” provee una respuesta canónica para el caso en que la recuperación no devuelva fragmentos pertinentes, evitando que el modelo invente para llenar el silencio.
- **Multi-chunk reasoning:** la instrucción “Combina fragmentos si es necesario para dar una respuesta completa” habilita explícitamente el razonamiento sobre múltiples piezas de contexto, en lugar de copiar literalmente un solo fragmento.
- **Lista numerada de reglas:** el bloque `REGLAS:` con *bullets* mejora el cumplimiento empírico frente a la misma información expresada en prosa libre.

XV-C2. SUMMARY_PROMPT:

- **Structured output / checklist:** enumera explícitamente las cinco dimensiones que debe cubrir el resumen (identidad, actividad, ubicaciones, valores, presencia global). El modelo las trata como una *checklist* a satisfacer.
- **Length constraint:** el límite “Máximo 300 palabras” es un control cuantitativo del *output* útil para integrar el resumen en interfaces con espacio acotado.
- **Goal framing:** “resumen ejecutivo claro y profesional” fija el género del texto esperado.

XV-C3. FAQ_PROMPT:

- **Output format template:** se especifica literalmente el formato de salida esperado:

```
**P: [pregunta]**
R: [respuesta basada en el documento]
```

 Esta plantilla literal es una forma ligera de *few-shot* sin ejemplos resueltos, suficiente para que el modelo replique la estructura.
- **Numerical constraint:** “exactamente 10 preguntas frecuentes” fuerza una cantidad determinista de elementos, evitando salidas con cantidades arbitrarias.
- **Topic diversity hint:** “Cubre temas variados: historia, productos, ubicaciones, sostenibilidad, contacto” guía la diversidad temática de las FAQs, evitando que el modelo se concentre en un único subtema.

- **User-perspective framing:** “que un cliente nuevo haría” hace que el modelo simule el punto de vista de un usuario externo, técnica útil para generar contenido orientado a personas que aún no conocen la empresa.

XV-D. Síntesis

Cuadro X
TÉCNICAS DE *prompt engineering* POR PROMPT.

Técnica	QA	Summary	FAQ
<i>Zero-shot</i> (sin ejemplos)	✓	✓	✓
Role prompting	✓	✓	✓
Grounding al contexto	✓	✓	✓
Separación system/human	✓	✓	✓
Idioma forzado (español)	✓	✓	✓
<i>Negative constraints</i>	✓		
<i>Fallback</i> canónico	✓		
Multi-chunk reasoning	✓		
Structured checklist		✓	
Length constraint		✓	
Output format template			✓
Numerical constraint			✓
Topic diversity hint			✓
User-perspective framing			✓

La combinación de técnicas no es decorativa: cada una mitiga un fallo específico observado en sistemas de Q&A sobre LLMs (alucinación, desviación de tema, formato inconsistente, longitud descontrolada, silencio mal manejado). El diseño se justifica como un conjunto de *guardrails* estratificados.

XVI. DISEÑO DE PROMPTS Y MITIGACIÓN DE RIESGOS

XVI-A. Principio de Fundamentación Exclusiva

Ambas aplicaciones instruyen explícitamente al LLM a no usar conocimiento de preentrenamiento sobre la empresa. Esta decisión de diseño es fundamental porque:

- Los datos corporativos (fusiones, plantas, productos) cambian con frecuencia.
- El nombre “Smurfit Kappa” puede confundirse con otras filiales del grupo.
- La fusión con WestRock (2024) generó un cambio de nombre a “Smurfit Westrock” que los modelos entrenados antes de esa fecha desconocen.

XVI-B. Temperatura Cero

Ambas aplicaciones usan `temperature=0.0`, lo que hace la generación determinista. Para sistemas de Q&A corporativo esto es preferible a temperaturas mayores, ya que reduce la variabilidad de las respuestas y facilita la validación.

XVI-C. Manejo de Preguntas sin Respuesta

Si el contexto recuperado no contiene información suficiente, el LLM debe responder “Esa información no está disponible” en lugar de inventar datos. Esta cláusula está explícitamente codificada en los prompts del sistema.

XVII. DECISIONES DE DISEÑO Y JUSTIFICACIÓN

XVII-A. 100 % Local como Línea Base

La decisión de implementar una primera versión sin APIs externas (OpenAI, Anthropic, Cohere, etc.) responde a: consideraciones de privacidad de datos corporativos, eliminación de costos variables por token y disponibilidad garantizada sin dependencia de servicios de terceros.

XVII-B. Trade-off Soberanía vs. Calidad

La incorporación de la variante comercial (`app_rag_comercial.py`) no contradice el principio anterior, sino que lo complementa al permitir elegir el punto de operación según el caso de uso:

- **Datos altamente confidenciales** (información financiera, estratégica, personal): la versión local es la única opción aceptable, ya que ningún fragmento del KB sale del entorno controlado.
- **Demos, prototipos y materiales de comunicación pública** (resúmenes ejecutivos, FAQs basados en el sitio web): la versión comercial ofrece calidad notoriamente superior y vale la pena su costo marginal.
- **Despliegue mixto:** para clientes que requieran lo mejor de ambos mundos, es viable un *router* que envíe consultas sensibles al modelo local y consultas públicas al modelo comercial, usando metadatos del KB como criterio de enrutamiento.

XVII-C. FAISS vs. Bases de Datos Vectoriales en la Nube

FAISS (Facebook AI Similarity Search) ofrece búsqueda aproximada de vecinos más cercanos en memoria RAM con rendimiento comparable a soluciones como Pinecone o Weaviate, pero sin requerir conectividad a Internet ni gestión de instancias. Para el volumen de datos del KB (<1 M vectores), FAISS es la opción más eficiente.

XVII-D. LangChain como Orquestador

LangChain provee abstracciones estables para el pipeline RAG: splitters, embeddings, vector stores y cadenas LCEL (*LangChain Expression Language*). Esto permite reemplazar cualquier componente (e.g., cambiar FAISS por Chroma) con mínimas modificaciones de código.

XVII-E. Encabezado Estático en el KB

La inyección manual de datos clave al inicio del KB garantiza que hechos corporativos fundamentales (fechas, plantas, cifras) estén siempre disponibles sin depender de que el scraper los haya indexado correctamente. Es una capa de seguridad sobre los datos scrapeados.

XVIII. ESTRUCTURA DE ARCHIVOS DEL PROYECTO

*Parte II — Evolución a Agente Conversacional

XIX. MOTIVACIÓN DE LA FASE 2

La Fase 1 dejó un sistema funcional pero con limitaciones claras: cada pregunta era un evento aislado (sin memoria), todas las respuestas se generaban desde la misma fuente (RAG), y la única forma de obtener un dato puntual (un teléfono, un NIT) era buscarlo dentro del texto narrativo del *knowledge base*.

La Fase 2 evoluciona el sistema en cuatro ejes:
leftmargin=2cm

1. **Memoria conversacional:** el asistente recuerda turnos previos y puede resolver referencias (“háblame de sus productos” → “¿dónde fabrican el primero?”).

```
proyecto/
|-- scraper.py           # Crawler async del sitio web (entrada del pipeline)
|-- build_kb.py         # Generacion del Knowledge Base a partir del web dump
|-- app.py              # Q&A lexico (Streamlit + Ollama)
|-- app_rag.py          # Q&A RAG (LangChain + FAISS + Ollama)
|-- app_rag_comercial.py # Q&A RAG con APIs comerciales (Gemini + Kimi K2.6 + FAISS)
'-- output/
    |-- full_dump.json   # Volcado completo del sitio (salida del scraper)
    |-- pages.jsonl      # Chunks preliminares (un JSON por linea)
    |-- url_index.txt     # Indice legible de URLs scrapeadas
    |-- scrape_log.txt    # Log de ejecucion del crawler
    |-- knowledge_base.md # KB compacta (6k palabras)
    |-- knowledge_base_rag.md # KB completa (sin limite)
    |-- faiss_index/      # Indice vectorial (embeddings Ollama)
    |   |-- index.faiss
    |   '-- index.pkl
    '-- faiss_index_gemini/ # Indice vectorial (embeddings Gemini)
        |-- index.faiss
        '-- index.pkl
```

Figura 4. Estructura de archivos completa del proyecto.

- 2. **Múltiples fuentes con router automático:** el LLM decide entre consultar el RAG documental o una base de datos estructurada (JSON curado), según la naturaleza de la pregunta.
- 3. **Interfaz tipo ChatGPT:** chat con burbujas, streaming visible, historial por usuario, en lugar de la interfaz Streamlit del módulo 1.
- 4. **Despliegue en producción:** el sistema deja de ser un prototipo local y queda accesible públicamente en `llm.innovatec.co`.

docstring extenso que el LLM lee como parte de su contexto al decidir cuál invocar.

XXI-A. *Herramienta 1: search_knowledge_base (RAG)*

Es la herramienta de información **no estructurada**. Reutiliza el índice FAISS construido en la Fase 1 con embeddings de Gemini:

XX. ARQUITECTURA DEL AGENTE

El agente se construye con `create_tool_calling_agent` de `LangChain`, que envuelve un LLM compatible con *function calling* (OpenAI, Moonshot, OpenRouter) y le proporciona descripciones formales de las herramientas disponibles. El `AgentExecutor` ejecuta el bucle clásico de razonamiento:

XX-A. *Componentes principales*

Cuadro XI
STACK TECNOLÓGICO DE LA FASE 2.

Capa	Tecnología
Backend API	FastAPI + Uvicorn
Orquestación	LangChain <code>create_tool_calling_agent</code>
LLM Comercial	OpenAI <code>gpt-5.1</code>
LLM Open-Source	OpenRouter <code>qwen/qwen3.5-9b</code>
Embeddings (RAG)	Gemini <code>gemini-embedding-2-preview</code>
Vector Store	FAISS (reutilizado de la Fase 1)
Persistencia	PostgreSQL con connection pool
Streaming	Server-Sent Events (SSE)
Observabilidad	LangSmith
Frontend	React 18 + Vite + TailwindCSS
Empaquetado	Dockerfile multi-stage (Node + Python)
Despliegue	Easypanel (VPS)

XXI. HERRAMIENTAS (Tools) DEL AGENTE

El agente dispone de dos herramientas registradas con el decorador `@tool` de `LangChain`. Cada una incluye un

Listing 12. Cuerpo de la herramienta RAG.

```
1 @tool
2 def search_knowledge_base(query: str) -> str:
3     """Busca en la base de conocimiento ...
4         documental
5         mediante recuperacion semantica (RAG).
6
7         Usar para preguntas abiertas sobre ...
8         productos,
9         historia, sostenibilidad, procesos, etc.
10        NO usar para datos puntuales como NIT, ...
11        telefonos,
12        direcciones para eso usar ...
13        get_company_info.
14
15        """
16    vs = _get_vectorstore()
17    docs = vs.similarity_search(query, k=4)
18    return "\n\n---\n\n".join(d.page_content ...
19                               for d in docs)
```

XXI-B. *Herramienta 2: get_company_info (datos estructurados)*

Es la herramienta de información **estructurada y determinística**. Lee de un archivo `company_info.json` curado manualmente con los datos puntuales más solicitados: razón social, NIT, teléfonos por sede, horarios, direcciones, certificaciones por planta, productos principales y datos históricos numéricos.

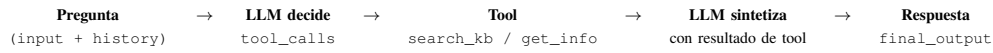


Figura 5. Bucle de ejecución del AgentExecutor en cada turno.

Listing 13. Fragmento de la herramienta estructurada.

```

1 @tool
2 def get_company_info(query: str) -> str:
3     """Recupera datos estructurados oficiales.
4
5     USAR para NIT, telefonos, correos, horarios,
6     direcciones, empleados, certificaciones, ...
7     de fundacion. NO usar para preguntas ...
8     abiertas.
9
10    """
11    q = query.lower()
12    result = {}
13    if "telefono" in q or "contacto" in q:
14        result["telefono"] = ...
15        COMPANY["telefono_principal"]
16        result["telefonos_por_sede"] = {...}
17    if "horario" in q:
18        result["horarios"] = ...
19        COMPANY["horarios_atencion"]
20    # ... y otros patrones similares
21    return json.dumps(result, ...
22                      ensure_ascii=False)
  
```

XXI-C. Importancia del docstring

El *docstring* de cada función actúa como **schema** que el LLM ve en cada request. Si la descripción de la herramienta es vaga, el router elige mal con frecuencia. Por eso el patrón consistente en ambas tools es:

- Una línea explicando **qué hace**.
- Una lista de casos donde **sí se debe usar**.
- Una negación explícita de cuándo **no usarla**, redirigiendo a la otra herramienta.

Este patrón reduce la ambigüedad y mejora el ratio de routing correcto en pruebas empíricas.

XXII. EL Meta-Prompt DEL AGENTE

El `SYSTEM_PROMPT` del agente es la pieza más densa en técnicas de *prompt engineering* de todo el proyecto. Tiene siete bloques, cada uno con un propósito específico de mitigación de fallos.

XXII-A. Estructura del prompt

Cuadro XII

BLOQUES DEL META-PROMPT DEL AGENTE Y TÉCNICAS QUE APLICAN.

Bloque	Técnica / propósito
Rol y audiencia	<i>Role prompting</i>
Descripción de cada tool	<i>Tool documentation</i> (refuerza el <i>docstring</i>)
Proceso (5 pasos)	<i>Chain-of-thought</i> obligatorio
5 ejemplos de routing	<i>Few-shot</i> de decisión (no de respuesta)
Formato de respuesta	<i>Output formatting</i> por tipo de pregunta
Restricciones absolutas	<i>Negative constraints / guardrails</i>
Fallback canónico	Respuesta literal cuando no hay datos

XXII-B. Cadena de razonamiento (*chain-of-thought*)

El bloque “Proceso para responder” obliga al modelo a razonar paso a paso antes de actuar:

1. **Clasificar** la pregunta (dato puntual / abierta / ambas / conversacional).
2. **Resolver referencias** del historial (“ese”, “el primero”, “ahí”) antes de llamar a la herramienta.
3. **Llamar** a la herramienta con un *query* específico.
4. **Sintetizar** la respuesta a partir del resultado.
5. **Fallback canónico** si la herramienta no devuelve datos: “Esa información no está disponible en la documentación oficial. Te recomiendo contactar...”

El paso 2 es lo que materializa la **memoria conversacional**: sin esa instrucción explícita, modelos pequeños tienden a llamar a la tool con un *query* literal del usuario (e.g. “el primero”), produciendo resultados inútiles.

XXII-C. Few-shot de routing en lugar de few-shot de respuesta

El prompt incluye cinco ejemplos de decisión:

- Ejemplo A: dato puntual “¿Cuál es el NIT?” → Tool 1.
- Ejemplo B: pregunta abierta “¿Qué es la cartulina Óptima?” → Tool 2.
- Ejemplo C: referencia con historial “¿Dónde fabrican el primero?” → resolución previa + Tool 1.
- Ejemplo D: pregunta multi-tool “¿Qué hacen en la planta de Cali y dónde queda?” → ambas tools.
- Ejemplo E: conversacional “Hola” → sin tools.

Estos ejemplos enseñan al modelo el **patrón de razonamiento**, no las respuestas concretas. El sistema sigue siendo *zero-shot* en cuanto a salidas: nunca se le muestran respuestas resueltas. Esta distinción es importante para defender que el prompt no es un *few-shot* disfrazado.

XXIII. MEMORIA CONVERSACIONAL PERSISTENTE

XXIII-A. Esquema PostgreSQL

Listing 14. Esquema de las dos tablas principales.

```

1 CREATE TABLE chats (
2     id          UUID PRIMARY KEY,
3     user_id     TEXT NOT NULL DEFAULT 'antonio',
4     title       TEXT NOT NULL,
5     created_at  TIMESTAMPTZ NOT NULL DEFAULT ...
6                 NOW(),
7     updated_at  TIMESTAMPTZ NOT NULL DEFAULT ...
8                 NOW()
9 );
10
11 CREATE TABLE messages (
12     id          UUID PRIMARY KEY,
13     chat_id     UUID NOT NULL REFERENCES ...
14                 chats(id)
15     role        TEXT NOT NULL
16                 CHECK (role IN ('user', ...
17                             'assistant', 'tool')),
18     content     TEXT NOT NULL,
19     tool_used    TEXT,
20     created_at  TIMESTAMPTZ NOT NULL DEFAULT ...
21                 NOW()
22 );
  
```

El campo `tool_used` permite auditar a posteriori qué herramienta eligió el agente en cada respuesta — útil para mostrar un *badge* en la UI y para evaluar la calidad del routing en pruebas.

XXIII-B. *Connection pool*

Las conexiones a PostgreSQL pasan por un `ConnectionPool` de `psycopg3`, que mantiene siempre dos conexiones abiertas y escala hasta diez bajo carga. Esto evita el *handshake* TCP+TLS por cada *request* (que en un VPS remoto añadía 150–400 ms a cada llamada a la base de datos).

XXIII-C. *Inyección del historial en el agente*

En cada turno, el handler de FastAPI recupera el historial completo de la conversación, lo convierte a tipos `HumanMessage` / `AIMessage` de `LangChain`, y lo inyecta en el prompt mediante un `MessagesPlaceholder("chat_history")`:

Listing 15. Inyección del historial en el `ChatPromptTemplate`.

```
1 prompt = ChatPromptTemplate.from_messages([
2     ("system", SYSTEM_PROMPT),
3     MessagesPlaceholder("chat_history"), # ...
4     ("human", "{input}"),
5     MessagesPlaceholder("agent_scratchpad"), ...
6     # <- razonamiento
7 ])
```

No se usa ningún resumen automático: se envía el historial completo de cada chat. Para el alcance del taller (conversaciones de pocos turnos) es suficiente. Si se proyectara a conversaciones largas, habría que implementar `ConversationSummaryMemory` o ventanas deslizantes.

XXIV. *Streaming* CON SERVER-SENT EVENTS

El endpoint `POST /api/chats/{id}/messages/stream` devuelve una respuesta `text/event-stream` con cuatro tipos de eventos en tiempo real, que la UI consume para mostrar el progreso del agente:

Cuadro XIII
EVENTOS DEL PROTOCOLO SSE ENTRE BACKEND Y FRONTEND.

Evento	Payload
<code>tool_start</code>	<code>{tool: "search_knowledge_base"}</code>
<code>tool_end</code>	<code>{tool: "search_knowledge_base"}</code>
<code>token</code>	<code>{content: ".E1"}</code> (uno por token)
<code>done</code>	<code>{answer, tools_used}</code>

Internamente se usa `agent.astream_events(version="v2")` de `LangChain`, que expone eventos del bucle del agente sin que tengamos que instrumentarlo manualmente. Filtramos por `on_tool_start`, `on_tool_end` y `on_chat_model_stream` para emitir cada uno como un evento SSE.

En la UI, esto se traduce en:

- Un *badge* “Consultando datos estructurados...” aparece cuando `tool_start` llega.

- El *badge* se vuelve verde con check al recibir `tool_end`.
- Una burbuja de respuesta se va llenando token a token (efecto de máquina de escribir).
- Un cursor parpadeante al final mientras siguen llegando tokens.

XXV. SISTEMA DE *Fallback* ESCALONADO

Un hallazgo empírico relevante es que los modelos pequeños (Qwen 3.5 9B en particular) ocasionalmente **llaman a las herramientas pero no producen la respuesta final**, especialmente en consultas multi-tool con historial. El usuario veía una burbuja vacía.

Para garantizar que siempre haya respuesta visible, el sistema implementa **cuatro niveles de respaldo** en cascada:

Cuadro XIV
NIVELES DE FALLBACK DEL AGENTE.

Nivel	Fuente del texto
1	Tokens de <code>on_chat_model_stream</code> (caso normal)
2	<code>output[.output]</code> del <code>AgentExecutor</code> en <code>on_chain_end</code>
3	Contenido del último <code>AIMessage</code> en <code>on_chat_model_end</code>
4	Llamada manual de síntesis: si las tools devolvieron datos pero el agente no respondió, se hace una llamada directa al LLM (sin agente, sin tool-calling) pasándole los outputs de las tools para que redacte.

XXV-A. *El nivel 4: recovery por síntesis manual*

Es la pieza más ingeniosa del sistema. En lugar de mostrar un mensaje de error, aprovechamos que las herramientas **ya devolvieron datos** (los capturamos durante el *stream*). Construimos un *prompt* mucho más simple, sin tool-calling, solo pidiendo síntesis:

Listing 16. Prompt de *recovery*.

```
1 synthesis_messages = [
2     ("system",
3      "Eres un asistente de Smurfit Westrock. ...
4      Te paso "
5      "la pregunta del usuario y los ...
6      resultados que ya "
7      "devolvieron las herramientas. Tu tarea ...
8      es UNICA-"
9      "MENTE redactar la respuesta final en ...
10     "español."),
11     ("human",
12      f"Pregunta: {user_input}\n\n"
13      f"Resultados de las ...
14      herramientas:\n{tools_block}\n\n"
15      f"Redacta la respuesta final."),
16 ]
17 msg = await llm.ainvoke(synthesis_messages)
```

Este prompt secundario es “trivial” para el modelo: no tiene que razonar sobre routing ni decidir tools, solo redactar a partir de datos proporcionados. Modelos pequeños lo resuelven sin problema. El resultado final es que el usuario siempre ve una respuesta coherente, incluso cuando el agente *per se* fallaría.

XXVI. MIGRACIÓN DEL MODELO LOCAL A OPENROUTER

Durante el desarrollo se usó ChatOllama apuntando a un proceso local de Ollama corriendo Qwen 3.5 9B. Al migrar a producción, el VPS no dispone de GPU, lo que hace inviable Qwen 9B en CPU (10–30× más lento).

La solución fue migrar a OpenRouter, un agregador que expone más de 200 modelos open-source y comerciales a través de una API OpenAI-compatible. El mismo modelo Qwen 3.5 9B queda accesible vía:

Listing 17. Inicialización del LLM “local” contra OpenRouter.

```
1 return ChatOpenAI(
2     model="qwen/qwen3.5-9b",
3     base_url="https://openrouter.ai/api/v1",
4     api_key=os.getenv("OPENROUTER_API_KEY"),
5     temperature=0.2,
6     streaming=True,
7     default_headers={
8         "HTTP-Referer": ...,
9         "https://llm.innovatec.co",
10        "X-Title": "Smurfit Westrock Asistente",
11    },
12 )
```

El cambio fue mínimo: solo `base_url` y `api_key`. No se modificó el agente, ni las herramientas, ni el prompt. Esto valida empíricamente la portabilidad de LangChain como capa de abstracción.

XXVI-A. ¿Sigue siendo “open-source”?

Sí. La etiqueta “open-source” se refiere a la **licencia del modelo**, no a su hosting. Qwen 3.5 sigue siendo Apache 2.0, y podríamos cambiar el hosting a uno *self-hosted* (Ollama, vLLM, TGI) modificando solamente las dos variables anteriores.

XXVII. EMPAQUETADO Y DESPLIEGUE

XXVII-A. Dockerfile multi-stage

El sistema completo (backend FastAPI + frontend React buildeado) se empaqueta en una sola imagen Docker. La primera etapa compila el frontend con Node, la segunda instala el backend en Python y copia los artefactos del frontend como archivos estáticos:

Listing 18. Dockerfile *multi-stage* (simplificado).

```
1 # Etapa 1: build del frontend
2 FROM node:20-alpine AS frontend-builder
3 WORKDIR /app
4 COPY frontend/package*.json ./
5 RUN npm ci
6 COPY frontend/ ./
7 RUN npm run build # genera /app/dist
8
9 # Etapa 2: backend con frontend embebido
10 FROM python:3.11-slim AS runtime
11 RUN apt-get install -y build-essential ...
12     libpq-dev curl
13 RUN pip install uv
14 WORKDIR /app
15 COPY backend/pyproject.toml backend/uv.lock ./
16 RUN uv sync --no-dev
```

```
16 COPY backend/ ./
17 COPY --from=frontend-builder /app/dist ./static
18 EXPOSE 8000
19 CMD ["uv", "run", "uvicorn", "main:app",
20     "--host", "0.0.0.0", "--port", "8000",
21     "--proxy-headers"]
```

XXVII-B. FastAPI sirviendo el SPA

FastAPI no solo expone la API REST: también actúa como servidor estático del bundle de Vite. Las rutas de la API se montan bajo `/api`, y un *catch-all* al final devuelve `index.html` para cualquier otra ruta, permitiendo el *client-side routing* del SPA:

Listing 19. Catch-all para servir el SPA buildeado.

```
1 api = APIRouter(prefix="/api")
2 # ... endpoints ...
3 app.include_router(api)
4
5 if STATIC_DIR.exists():
6     app.mount("/assets",
7         StaticFiles(directory=STATIC_DIR ...
8             / "assets"))
9
10 @app.get("/{full_path:path}")
11 async def serve_spa(full_path: str):
12     target = STATIC_DIR / full_path
13     if full_path and target.is_file():
14         return FileResponse(target)
15     return FileResponse(STATIC_DIR / ...
16         "index.html")
```

Beneficios de servir ambos desde el mismo proceso:

- **Same-origin**: el frontend hace `fetch('/api/...')` al mismo host. Cero problemas de CORS en producción.
- Un único contenedor a desplegar, monitorear y escalar.
- Un solo dominio, un solo certificado SSL.

XXVII-C. Easypanel como PaaS

El despliegue se hace en un VPS con **Easypanel**, una plataforma PaaS open-source con UI similar a Vercel/Heroku. El flujo es:

1. Push a la rama `main` del repo GitHub.
2. Easypanel detecta el push y clona el repo.
3. Construye el *Dockerfile multi-stage*.
4. Configura el reverse proxy hacia `llm.innovatec.co`.
5. Genera el certificado SSL automáticamente con Let's Encrypt.

Tiempo total del primer despliegue: ~5 minutos. Despliegues posteriores: ~3 minutos (aprovechando caché de capas Docker).

XXVIII. OBSERVABILIDAD CON LANGSMITH

LangSmith es la plataforma de observabilidad oficial de LangChain. Se activa simplemente definiendo tres variables de entorno (`LANGSMITH_TRACING=true`, `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT`). Sin tocar código, LangChain instrumenta cada Runnable automáticamente y envía cada ejecución como una traza.

Para cada conversación se puede inspeccionar:

- El árbol completo de la ejecución: AgentExecutor → ChatOpenAI (decisión inicial) → Tool → ChatOpenAI (síntesis).
- El *prompt* exacto enviado al LLM (incluyendo el system prompt, el historial y los ejemplos de *few-shot*).
- Los argumentos con que se llamó a cada herramienta y su salida.
- La latencia de cada paso.
- Los *tokens* consumidos y el costo en USD por traza.

Esta instrumentación es crítica para defender el sistema empíricamente: permite mostrar en vivo cómo el *meta-prompt* influye en la decisión de routing del modelo.

XXIX. INTERFAZ WEB REACTIVA

El frontend está construido con **React 18**, **Vite** como *bundler* y **TailwindCSS** para los estilos, con una paleta azul acorde a la identidad de Smurfit Westrock. Componentes principales:

Cuadro XV
COMPONENTES DE LA UI DEL MÓDULO 2.

Componente	Función
ModelSelect	Pantalla inicial: elegir comercial o open-source
Login	Selección de usuario (4 avatares clickeables)
Sidebar	Historial de conversaciones del usuario activo
ChatWindow	Lista de mensajes con auto-scroll y skeleton loader
MessageBubble	Burbuja individual con <i>badge</i> de tool usada
ThinkingIndicator	“Pensando...” + indicador de tool en curso
ChatInput	Textarea con auto-resize, envío con Enter
SettingsModal	Sliders para <i>temperature</i> , <i>top_p</i> , etc.

La preferencia de modelo, el usuario activo y los parámetros de muestreo se persisten en *localStorage* con claves distintas por proveedor (*sw_settings_commercial*, *sw_settings_local*), de modo que los ajustes hechos sobre GPT-5.1 no afectan los de Qwen y viceversa.

XXX. CASOS DE PRUEBA Y VALIDACIÓN DEL AGENTE

Para demostrar que cada capacidad funciona, el sistema se valida con cuatro tipos de pregunta:

Cuadro XVI
CASOS DE PRUEBA PARA VALIDAR LAS CAPACIDADES DEL AGENTE.

Caso	Pregunta de ejemplo	Tool esperada
RAG documental	¿Qué es la cartulina Óptima y para qué sirve?	<i>search_knowledge_base</i>
Dato estructurado	¿Cuál es el NIT de la empresa?	<i>get_company_info</i>
Memoria	“Háblame de sus productos” → “¿Cuál mencionaste primero?”	RAG + resolución por contexto
Multi-tool	¿Qué hacen en la planta de Cali y dónde queda?	ambas tools

En todos los casos validamos no solo que la respuesta sea correcta, sino que el *badge* mostrado en la UI corresponda a la herramienta adecuada. LangSmith adicionalmente permite verificar a posteriori que el LLM razonó por el camino esperado.

XXXI. DIFERENCIAS CONCEPTUALES ENTRE FASES

Cuadro XVII
EVOLUCIÓN DEL SISTEMA ENTRE FASE 1 Y FASE 2.

Capacidad	Fase 1	Fase 2
Memoria conversacional	No (preguntas aisladas)	Sí (Postgres)
Fuentes de info	Una (RAG)	Dos (RAG + estructurada) con router
Selector de modelo	Editar código	UI (comercial / open-source)
Usuarios	Único	4 usuarios con historial separado
UI	Streamlit	React con streaming
Empaquetado	Local con uv	Docker <i>multi-stage</i>
Despliegue	Localhost	VPS público con SSL
Observabilidad	Logs de Streamlit	LangSmith con trazas
Visibilidad del razonamiento	Solo respuesta final	<i>Badges</i> de tools + streaming

XXXII. CONCLUSIONES

El sistema implementado demuestra que es posible construir un asistente virtual corporativo robusto cubriendo todo el espectro entre soberanía total de datos (versiones 100 % locales) y calidad de modelo de frontera (versión basada en APIs comerciales), usando un mismo conjunto de abstracciones de orquestación. Los aspectos más destacables del diseño son:

1. **Fundamentación estricta:** los prompts prohíben explícitamente el uso de conocimiento externo, garantizando respuestas basadas en fuentes oficiales.
2. **Arquitectura triple:** la coexistencia de un motor léxico ligero, un motor RAG semántico local y un RAG semántico con APIs comerciales permite adaptar el sistema a diferentes restricciones de hardware, confidencialidad y exigencia de calidad sin rediseñar la solución.
3. **Portabilidad multi-proveedor:** el uso de LangChain como orquestador permite intercambiar embeddings y LLMs entre Ollama (local) y proveedores comerciales (Google Gemini, Moonshot AI Kimi K2.6) modificando únicamente la inicialización de los componentes; la cadena LCEL permanece idéntica.
4. **Manejo robusto de *rate limits*:** la versión comercial incorpora reintento con *exponential backoff* y *batching* manual, lo que la hace operativa tanto en *Free Tier* como en *Paid Tier* de los proveedores.
5. **Persistencia del índice:** el índice FAISS se genera solo una vez y se reutiliza en ejecuciones posteriores, haciendo el arranque casi inmediato a partir de la segunda ejecución. Cada variante mantiene su propio índice para evitar mezclar espacios vectoriales incompatibles.
6. **Preprocesamiento cuidadoso del KB:** la limpieza de ruido de navegación, la deduplicación y el encabezado estático curado manualmente mejoran significativamente la calidad de los fragmentos disponibles para recuperación, beneficiando por igual a las tres variantes.
7. **Extensibilidad:** la interfaz de pestañas y el uso de LangChain como orquestador facilitan agregar nuevas funcionalidades (e.g., agentes, memoria conversacional) con esfuerzo mínimo.

Como trabajo futuro se identifican: la adición de memoria conversacional para preguntas de seguimiento; la integración

de evaluación automática de la relevancia de los *chunks* recuperados; la exploración de modelos de embeddings multilingües para soportar consultas en inglés; una evaluación cuantitativa comparando las respuestas de la variante local frente a la comercial mediante métricas como BERTScore y evaluación humana; un *router* dinámico que escoja entre LLM local o comercial según la confidencialidad o complejidad de la consulta; y un mecanismo de *caché* de respuestas comerciales para reducir costos en preguntas recurrentes.